

FastAPI Notes

Concept x MEDSCAN.AI Usage

Side-by-side explanations from scratch

Project:	MEDSCAN.AI - Disease Risk Predictor
Stack:	FastAPI / SQLAlchemy / JWT / Celery / Redis
Covers:	18 core FastAPI concepts

Generated automatically - `python generate_fastapi_notes.py`

1. What is FastAPI?

FastAPI is a modern Python web framework for building APIs. It auto-generates interactive documentation at /docs, validates all inputs via Python type hints and Pydantic, handles both sync and async route functions, and produces one of the fastest Python API servers available.

General Concept	In MEDSCAN.AI
Framework for building REST APIs in Python. Auto-docs, validation, async support.	The entire backend/ directory is one FastAPI app. It handles file uploads, report results, JWT auth, and kicks off Celery ML tasks.

2. Creating the App

General syntax

General

```
from fastapi import FastAPI
app = FastAPI(title="My App")
```

MEDSCAN.AI usage

backend/app/main.py

```
# backend/app/main.py
app = FastAPI(title=settings.PROJECT_NAME)
# settings.PROJECT_NAME reads MEDSCAN_PROJECT_NAME from env var
# The title shows up in the Swagger UI at /docs
```

3. Routes / Path Operations

A decorator (@app.get, @app.post, etc.) registers a function as the handler for a specific HTTP method + URL path. FastAPI calls the function when a matching request arrives.

General

```
# General
@app.get("/hello")
def say_hello():
    return {"message": "Hello"}
```

backend/app/main.py

```
# Health-check route (backend/app/main.py)
@app.get("/")
async def root():
    return {"message": "Welcome to Disease Risk Predictor API"}
# Render pings / every 30 s to confirm the container is alive
```

4. async def vs def

Keyword	What it means
async def	Can use await for async I/O (database, HTTP calls). FastAPI runs it in the async event loop.
def	Synchronous function. FastAPI runs it in a thread pool so it does not block the event loop.

In MEDSCAN.AI: most API routes use async def. Auth routes (register, login, refresh) use plain def because they use synchronous SQLAlchemy sessions. FastAPI handles both automatically.

5. Path Parameters

Curly-brace segments in the URL path become typed function parameters. FastAPI validates and converts the value automatically. Passing a non-integer when int is expected returns 422 automatically.

General

```
# General
@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {"id": item_id}
```

backend/app/api/status.py

```
# Status polling endpoint (backend/app/api/status.py)
@router.get("/status/{report_id}")
async def get_report_status(report_id: int, ...):
    report = db.query(Report)
        .filter(Report.id == report_id,
                Report.user_id == current_user.id).first()
# Frontend calls GET /api/status/42
# "42" arrives as Python int - FastAPI converts it automatically
```

6. Query Parameters

Any parameter NOT in the path {} is treated as a query parameter: /items?skip=5&limit=20. Default values make them optional.

General

```
# General
@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    ...
# Called as: GET /items?skip=5&limit=20
```

MEDSCAN.AI does not use query params heavily. All filtering uses the authenticated `current_user.id` inside the route handler, so GET `/api/reports` always returns only that user's data.

7. Request Body (Pydantic Models)

FastAPI reads the JSON request body and validates it against a Pydantic BaseModel. Requests that do not match the schema are rejected with 422 Unprocessable Entity before your function even runs.

General

```
# General
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float

@app.post("/items")
def create_item(item: Item):
    return item
```

backend/app/api/compare.py

```
# Compare trigger (backend/app/api/compare.py)
class CompareRequest(BaseModel):
    report1_id: int
    report2_id: int

@router.post("/compare")
async def trigger_comparison(body: CompareRequest, ...):
    r1 = db.query(Report).filter(Report.id == body.report1_id).first()

# Frontend sends: { "report1_id": 3, "report2_id": 7 }
# FastAPI validates both are integers before the function runs
```

8. Form Data

Form(...) reads a value from application/x-www-form-urlencoded or multipart/form-data bodies. The ... means the field is required.

General

```
# General
from fastapi import Form

@app.post("/login")
def login(username: str = Form(...), password: str = Form(...)):
    ...
```

backend/app/auth/router.py + api/upload.py

```
# Login uses OAuth2 standard form format (auth/router.py)
from fastapi.security import OAuth2PasswordRequestForm

@router.post("/login")
def login(form_data: OAuth2PasswordRequestForm = Depends(), ...):
    user = db.query(User)
        .filter(User.username == form_data.username).first()

# Upload also uses Form for the type string alongside the file
report_type: str = Form(...) # same multipart request as File(...)
```

9. File Uploads

UploadFile gives you the file-like object, the original filename, and the content type. List[UploadFile] accepts multiple files in one request.

General

```
# General
from fastapi import UploadFile, File

@app.post("/upload")
async def upload(file: UploadFile = File(...)):
    content = await file.read()
```

backend/app/api/upload.py

```
# Batch upload (backend/app/api/upload.py)
@router.post("/upload")
async def upload_reports(
    files: List[UploadFile] = File(...), # one or more files
    report_type: str = Form(...),
    ...
):
    for file in files:
        uid = f"{current_user.id}_{os.urandom(8).hex()}{ext}"
        s3.upload_fileobj(file.file, bucket, s3_key)
# file.file is the raw file-like object passed directly to boto3
```

10. Dependency Injection (Depends)

A "dependency" is a function that FastAPI calls automatically before your route, injects its return value, and handles cleanup. Dependencies can chain into each other. This is FastAPI's most powerful feature for shared logic like auth and DB sessions.

General

```
# General - database session dependency
def get_db():
    db = SessionLocal()
    try:
        yield db          # db is injected into the route
    finally:
        db.close()       # always cleaned up after route returns

@app.get("/items")
def list_items(db: Session = Depends(get_db)):
    return db.query(Item).all()
```

■ Three-layer dependency chain in MEDSCAN.AI

Layer 1 - DB session (database.py):

backend/app/database.py

```
def get_db():
    db = SessionLocal()    # sync psycopg2 session for Celery compat
    try:
        yield db
    finally:
        db.close()
```

Layer 2 - JWT decode (auth/dependencies.py):

backend/app/auth/dependencies.py

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")
# Automatically reads "Authorization: Bearer <token>" header

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> User:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    user = db.query(User)
        .filter(User.username == payload["sub"]).first()
    return user
```

Layer 3 - Active account check (chains to layer 2):

backend/app/auth/dependencies.py

```
def get_current_active_user(
    current_user: User = Depends(get_current_user),
) -> User:
    if not current_user.is_active:
        raise HTTPException(403, "Inactive user")
    return current_user
```

Used in every protected route:

backend/app/api/reports.py

```
@router.get("/reports")
async def list_reports(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_active_user),
):
    return db.query(Report)
        .filter(Report.user_id == current_user.id).all()

# FastAPI runs automatically:
# get_db -> oauth2_scheme -> get_current_user
#         -> get_current_active_user -> list_reports()
```


11. HTTPException - Returning Errors

Raise `HTTPException` anywhere in your route or dependency to immediately return a JSON error response. FastAPI catches it and sends the correct status code with `{"detail": "..."} body`.

General

```
# General
from fastapi import HTTPException

raise HTTPException(status_code=404, detail="Item not found")
# Returns HTTP 404: {"detail": "Item not found"}
```

backend/app/api/compare.py

```
# Three validation checks in one route (api/compare.py)

if not r1 or not r2:
    raise HTTPException(404, "One or both reports not found")

if r1.report_type != r2.report_type:
    raise HTTPException(400, "Reports must be the same type")

if r1.status != "completed" or r2.status != "completed":
    raise HTTPException(400, "Both reports must be fully processed")
```

12. APIRouter - Splitting Routes into Files

Instead of registering all routes on app directly, each feature area defines its own router = `APIRouter()`. `main.py` assembles them all with `include_router()` and a URL prefix.

General

```
# In a feature file (items.py)
from fastapi import APIRouter
router = APIRouter()

@router.get("/items")
def list_items(): ...

# In main.py
from items import router
app.include_router(router, prefix="/api")
# Routes become /api/items
```

backend/app/main.py

```
# MEDSCAN.AI - five routers assembled in main.py
app.include_router(upload.router, prefix="/api", tags=["upload"])
app.include_router(status.router, prefix="/api", tags=["status"])
app.include_router(reports.router, prefix="/api", tags=["reports"])
app.include_router(compare.router, prefix="/api", tags=["compare"])
app.include_router(auth_router) # has prefix="/auth" inside its file

# Each file: router = APIRouter()
# auth/router.py: router = APIRouter(prefix="/auth", tags=["auth"])
# tags= groups routes in the /docs Swagger UI
```

13. Response Models

`response_model=` tells FastAPI to filter the return value through a Pydantic schema before sending it. Fields not in the schema are stripped - useful for hiding sensitive fields like password hashes.

General

```
# General - strip password from response
class UserOut(BaseModel):
    id: int
    email: str
    # no password field

@app.post("/users", response_model=UserOut)
def create_user(user_in: UserCreate) -> UserOut:
    user = User(email=user_in.email, password=hash(...))
    return user # hashed_password stripped from response
```

backend/app/auth/router.py

```
# MEDSCAN.AI - register (auth/router.py)
@router.post("/register", response_model=UserRead, status_code=201)
def register(...) -> UserRead:
    user = User(
        username=user_in.username,
        hashed_password=get_password_hash(user_in.password),
    )
    db.add(user); db.commit(); db.refresh(user)
    return user
# hashed_password is NOT in UserRead schema -> stripped from response
```

14. Status Codes

Default response status is 200 OK. Use `status_code=` to override. Import `fastapi.status` for named constants instead of raw integers.

Status Code	When used in MEDSCAN.AI
201 Created	POST /auth/register - a new User row was created

Status Code	When used in MEDSCAN.AI
400 Bad Request	Wrong report type, reports not completed, username taken
401 Unauthorized	JWT missing, expired, or wrong type (access vs refresh)
403 Forbidden	Account is deactivated (is_active = False)
404 Not Found	Report/comparison ID does not exist or belongs to another user
422 Unprocessable	Pydantic validation failed - auto-generated by FastAPI
429 Too Many Req	Rate limit exceeded - handled by slowapi middleware
500 Server Error	DB error creating a row, or Celery queue failure

15. Middleware (CORS)

Middleware wraps every single request before it reaches a route. CORS (Cross-Origin Resource Sharing) middleware adds the headers the browser requires to allow JavaScript to call an API on a different domain or port.

General

```
# General
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

backend/app/main.py

```
# MEDSCAN.AI uses regex instead of a list (main.py)
# Reason: Vite dev server uses any port (5173, 5174 ...)
# A plain list would need a new entry for every port tried.
app.add_middleware(
    CORSMiddleware,
    allow_origin_regex=
        r"^https?://(localhost|127\.\0\.\0\.\1)(:\d+)?$"
        r"|^https://medscan-ai-s7t1\.onrender\.com$",
    allow_credentials=True, # required for Authorization header
    allow_methods=["*"],
    allow_headers=["*"],
)
```

16. Startup Events

Code decorated with `@app.on_event("startup")` runs once when Uvicorn starts, before any HTTP request is handled. Used for one-time setup like running database migrations.

General

```
# General
@app.on_event("startup")
def on_startup():
    create_tables() # set up DB schema before first request
```

backend/app/main.py

```
# MEDSCAN.AI - auto-migrate on every deploy (main.py)
@app.on_event("startup")
def on_startup():
    try:
        alembic_cfg = Config(str(BASE_DIR / "alembic.ini"))
        command.upgrade(alembic_cfg, "head") # all pending migrations
    except Exception as e:
        models.Base.metadata.create_all(bind=engine) # fallback

# Render restarts the container on every deploy.
# Auto-migrating on startup = zero-downtime schema updates.
```

17. Rate Limiting (slowapi)

FastAPI has no built-in rate limiting. The slowapi library hooks into the request flow and tracks request counts per IP per time window. The `@limiter.limit("N/period")` decorator is applied per route. The request: Request parameter is REQUIRED - slowapi reads the IP from it.

General

```
# General setup
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.post("/login")
@limiter.limit("5/minute")
def login(request: Request, ...): # request: Request is REQUIRED
    ...
```

backend/app/auth/router.py

```
# MEDSCAN.AI - different limits for three auth endpoints

# main.py - attach to app
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# auth/router.py - per-route limits
@router.post("/register")
@limiter.limit("3/minute")    # 3 registrations per IP per minute
def register(request: Request, ...):

@router.post("/login")
@limiter.limit("5/minute")    # 5 login attempts per IP per minute
def login(request: Request, ...):

@router.post("/refresh")
@limiter.limit("10/minute")    # 10 silent refreshes per IP per minute
def refresh(request: Request, ...):
# request: Request looks unused but slowapi reads the IP from it
```

18. Full Request Lifecycle in MEDSCAN.AI

How GET /api/reports flows through every FastAPI layer from the browser to the database and back:

- 1 **Browser sends the request**
GET /api/reports with "Authorization: Bearer <token>" header
- 2 **CORS Middleware**
Checks origin against allow_origin_regex. Adds Allow-Origin header to the response.
- 3 **slowapi Middleware**
Checks request count for this IP address. Returns 429 if over the per-minute limit.
- 4 **Router matches the route**
GET /api/reports matches list_reports() registered in reports.router
- 5 **Depends(get_db) runs**
Opens a synchronous psycopg2 SQLAlchemy session. Yielded into the route.
- 6 **Depends(get_current_active_user) chains**
oauth2_scheme extracts the JWT -> jwt.decode verifies signature + expiry -> db.query(User) loads the user row -> is_active checked
- 7 **list_reports() executes**
db.query(Report).filter(Report.user_id == current_user.id).order_by(Report.created_at.desc()).all()
- 8 **FastAPI serialises the response**
Returns 200 OK with a JSON array of report dicts
- 9 **Cleanup runs**
get_db finally block: db.close() - session returned to the connection pool

Every protected route follows this exact pattern. The dependency chain handles auth, DB session, and cleanup automatically - each route function only needs to contain business logic.

Quick Reference - All Routes

Route	Purpose
POST /auth/register	Create account - returns UserRead (no password)
POST /auth/login	Verify credentials - returns access + refresh JWT pair
POST /auth/refresh	Exchange refresh token for a new access token
POST /api/upload	Upload image(s) - enqueues Celery OCR + ML task
GET /api/status/{report_id}	Poll pipeline status - returns full result when done
GET /api/reports	List all user reports (History and Compare UI)
GET /api/reports/{id}	Full detail for one report including ML prediction
GET /api/reports/{id}/download	Stream file or 302 redirect to S3 presigned URL

Route	Purpose
POST /api/compare	Validate two reports and enqueue comparison task
GET /api/compare	List past comparisons for the current user
GET /api/compare/{id}	Poll comparison - returns structured diff when done
GET /	Health-check - Render pings this every 30 seconds